CS 4122: Reinforcement Learning

Programming Assignment #3

Optimizing Air Quality Monitoring using TD-Learning

Team Contributions

Name	Roll No.	Technical Contribution
Sushan	SE23UCSE073	**Section 6 (Custom Environment Core):** Designed the core `SensorTransmissionEnv` class and implemented the complex `step()` function, including the asymmetric loss function and battery transition logic. Formulated the official MDP documentation.
Ankit	SE23UCSE160	**Section 6 & Section 7 (Standard Q-Learning):** Contributed to the Custom Environment by implementing the state-dependent action space masking logic. Implemented the baseline Q-Learning algorithm (`train.py`), handling ϵ-greedy decay and the `Test.py` evaluation pipeline.
Nishal	SE23UARI084	**Section 6 & Section 7 (Structural Knowledge):** Contributed to the Custom Environment by implementing the initial state randomization in the `reset()` function and episode truncation. Implemented the Structural Knowledge Q-Learning algorithm and its pseudo-transition inference logic.
Nitin	SE23UCSE127	**Section 6 & Section 8 (Analysis):** Contributed to the Custom Environment by integrating the Markovian matrices (`air.npy`) and solar distributions (`solar.npy`) into the `__init__` state. Engineered the evaluation pipelines, generated comparative plots, and analyzed optimal behavioral patterns.

Contents

6 Coding Custom Environment: MDP Formulation **3**
6.1 State and State Space 3
6.2 Action and Action Space 3
6.3 Reward 4
6.4 Transition Equations 4
6.5 Transition Probabilities 5
6.6 Bellman Optimality Equation 6

7 Training RL Algorithms **6**
7.1 Structural Knowledge Cases 6

8 Analysis **8**
8.1 Algorithm Comparison 8
8.2 Pattern for Action 1 (Do Not Transmit) 8
8.3 Pattern for Action 3 (Transmit Max) 9

6 Coding Custom Environment: MDP Formulation

The problem is formulated as an infinite-horizon discounted Markov Decision Process. The objective of the sensor is to minimize the expected discounted loss

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \beta^t l_t\right], \tag{1}$$

where $\beta \in (0, 1)$ is the discount factor.

6.1 State and State Space

Let $\mathcal{S} = \{0, 0.02, 0.04, \ldots, 0.98, 1\}$ be the set of possible normalized air pollution values.

At the beginning of time slot t, the state is $x_t = (\theta_t, b_t, e_t, m_t)$. Here,

- $\theta_t \in \mathcal{S}$ is the true air pollution at time slot t.
- $b_t \in \{0, 1, \ldots, B\}$ is the battery level at time slot t.
- $e_t \in \mathcal{S}$ is the central station's estimate before the transmission result of time slot t. Therefore, $e_t = \hat{\theta}_{t-1}$.
- $m_t \in \mathcal{S}$ is the maximum air pollution observed since the last successful transmission, including the current value θ_t.

The variable m_t is required because action 3 transmits the maximum air pollution value since the last successful transmission. Thus, the state space is:

$$\mathcal{X} = \{(\theta, b, e, m) : \theta \in \mathcal{S}, b \in \{0, 1, \ldots, B\}, e \in \mathcal{S}, m \in \mathcal{S}, m \geq \theta\}. \tag{2}$$

6.2 Action and Action Space

At each time slot, the sensor can choose one of the following actions:

- $a_t = 1$: Do not transmit.
- $a_t = 2$: Transmit the true air pollution value θ_t.
- $a_t = 3$: Transmit the maximum air pollution value m_t since the last successful transmission.

Actions 2 and 3 are possible only if the battery level is at least η. Therefore, the state-dependent action space is:

$$\mathcal{A}(x_t) = \begin{cases} \{1\}, & b_t < \eta \\ \{1, 2, 3\}, & b_t \geq \eta \end{cases} \tag{3}$$

6.3 Reward

The assignment defines a loss function. Since reinforcement learning algorithms usually maximize reward, we define the reward as the negative of the loss.

The important point is that $\hat{\theta}_t$ is decided *after* the transmission attempt in time slot t. Therefore, the loss at time slot t is computed using the updated estimate $\hat{\theta}_t$, not the old estimate e_t. Define:

$$L(\theta, z) = \begin{cases} |\theta - z|, & \theta \leq z \\ 1.5 \cdot |\theta - z|, & \theta > z \end{cases} \tag{4}$$

Here, z is the estimate after the transmission attempt in the current time slot. The reward is $r_t = -L(\theta_t, \hat{\theta}_t)$. Equivalently, $r(x_t, a_t) = -c(x_t, a_t)$, where $c(x_t, a_t)$ is the expected immediate loss.

For action 1, no transmission occurs. Therefore, $\hat{\theta}_t = e_t$. Hence,

$$c(x_t, 1) = L(\theta_t, e_t). \tag{5}$$

For action 2, the sensor transmits θ_t. The transmission succeeds with probability λ. Therefore,

$$\hat{\theta}_t = \begin{cases} \theta_t, & \text{with probability } \lambda \\ e_t, & \text{with probability } 1 - \lambda \end{cases} \tag{6}$$

The expected immediate loss is $c(x_t, 2) = \lambda L(\theta_t, \theta_t) + (1 - \lambda) L(\theta_t, e_t)$. Since $L(\theta_t, \theta_t) = 0$, this becomes:

$$c(x_t, 2) = (1 - \lambda) L(\theta_t, e_t). \tag{7}$$

For action 3, the sensor transmits m_t. The transmission succeeds with probability λ. Therefore,

$$\hat{\theta}_t = \begin{cases} m_t, & \text{with probability } \lambda \\ e_t, & \text{with probability } 1 - \lambda \end{cases} \tag{8}$$

The expected immediate loss is:

$$c(x_t, 3) = \lambda L(\theta_t, m_t) + (1 - \lambda) L(\theta_t, e_t). \tag{9}$$

The expected reward is $r(x_t, a_t) = -c(x_t, a_t)$.

6.4 Transition Equations

Let the current state be $x_t = (\theta, b, e, m)$ and the next state be $x_{t+1} = (\theta', b', e', m')$.

The air pollution evolves according to the Markov transition probability:

$$Pr(\theta_{t+1} = \theta' | \theta_t = \theta) = p_{\theta, \theta'} \tag{10}$$

The solar energy process is iid. Let $\delta_t \in \{0, 1, \ldots, \Delta\}$ be the solar energy harvested

in time slot t, with $Pr(\delta_t = \delta) = \alpha_\delta$.

The battery transition is:

$$b' = \min(B, b - \eta I(a) + \delta), \tag{11}$$

where

$$I(a) = \begin{cases} 1, & a \in \{2, 3\} \\ 0, & a = 1 \end{cases} \tag{12}$$

The estimate transition is as follows.

If $a = 1$, no transmission occurs:

$$e' = e. \tag{13}$$

If $a = 2$, the sensor transmits θ:

$$e' = \begin{cases} \theta, & \text{with probability } \lambda \\ e, & \text{with probability } 1 - \lambda \end{cases} \tag{14}$$

If $a = 3$, the sensor transmits m:

$$e' = \begin{cases} m, & \text{with probability } \lambda \\ e, & \text{with probability } 1 - \lambda \end{cases} \tag{15}$$

The maximum pollution memory is updated based on whether there was a successful transmission. If the transmission in time slot t is successful, then:

$$m' = \theta'. \tag{16}$$

If there is no successful transmission, then:

$$m' = \max(m, \theta'). \tag{17}$$

6.5 Transition Probabilities

For action 1, no transmission occurs:

$$P(x'|x, 1) = p_{\theta,\theta'} \sum_{\delta=0}^{\Delta} \alpha_\delta \mathbf{1}\{b' = \min(B, b + \delta)\} \mathbf{1}\{e' = e\} \mathbf{1}\{m' = \max(m, \theta')\}. \tag{18}$$

For action 2, the sensor transmits θ. This action is valid only if $b \geq \eta$:

$$P(x'|x,2) = p_{\theta,\theta'} \sum_{\delta=0}^{\Delta} \alpha_\delta \mathbf{1}\{b' = \min(B, b-\eta+\delta)\} \Big[\lambda \mathbf{1}\{e' = \theta\}\mathbf{1}\{m' = \theta'\} + (1-\lambda)\mathbf{1}\{e' = e\}\mathbf{1}\{m' = \max(m,\theta')\}\Big]. \tag{19}$$

For action 3, the sensor transmits m. This action is valid only if $b \geq \eta$:

$$P(x'|x,3) = p_{\theta,\theta'} \sum_{\delta=0}^{\Delta} \alpha_\delta \mathbf{1}\{b' = \min(B, b-\eta+\delta)\} \Big[\lambda \mathbf{1}\{e' = m\}\mathbf{1}\{m' = \theta'\} + (1-\lambda)\mathbf{1}\{e' = e\}\mathbf{1}\{m' = \max(m,\theta')\}\Big]. \tag{20}$$

6.6 Bellman Optimality Equation

Since the objective is to minimize discounted loss, the optimal value function satisfies:

$$V^*(x) = \min_{a \in \mathcal{A}(x)} \left[c(x,a) + \beta \sum_{x' \in \mathcal{X}} P(x'|x,a) V^*(x') \right]. \tag{21}$$

Equivalently, using rewards,

$$V^*(x) = \max_{a \in \mathcal{A}(x)} \left[r(x,a) + \beta \sum_{x' \in \mathcal{X}} P(x'|x,a) V^*(x') \right], \tag{22}$$

where $r(x,a) = -c(x,a)$. Thus, the MDP is $(\mathcal{X}, \mathcal{A}, P, r, \beta)$.

7 Training RL Algorithms

7.1 Structural Knowledge Cases

In standard TD learning, we only update the Q-value for the state-action pair we physically visit. Structural Knowledge Q-Learning improves sample efficiency by using the current observed sample (x_1, a_1, r_1, x_1') to predict the reward and next state (r_2, x_2') for a completely different state-action pair (x_2, a_2). This allows us to update multiple Q-values per time slot.

While the exact transition probabilities (like the solar energy distribution α_δ and the channel success rate λ) are unknown, the deterministic mechanics of the system are known. To avoid incorrect training, we restrict our predictions to the following mathematically safe cases as discussed in the lecture material:

General Constraints for Prediction:

1. **Pollution Independence:** We can only predict outcomes for target states where the true pollution is identical to the observed sample ($\theta_2 = \theta_1$). We cannot predict transitions for other pollution levels because we do not know the underlying Markov transition matrix probabilities.

2. **The Battery Boundary Edge Case:** If the observed next battery level hits the maximum capacity ($b'_1 = B$), the exact solar energy δ harvested cannot be uniquely inferred (as any surplus energy is truncated). In such boundary cases, we cannot reliably calculate b'_2, so we skip updating any state where $b_2 \neq b_1$.

Case 1: Observed Action is No Transmission ($a_1 = 1$)

- **Predicting for $a_2 = 1$ (No Transmit):** We can successfully predict this. The battery delta is inferred as $\delta = b'_1 - b_1$. The target's next state will simply update the max memory ($m'_2 = \max(m_2, \theta'_1)$), leaving the estimate unchanged ($e'_2 = e_2$).

- **Predicting for $a_2 \in \{2, 3\}$ (Transmit):** This is **impossible**. Because the current sample did not transmit, we have no sample of the wireless channel's success/failure for this time slot. We cannot predict outcomes for transmission actions.

Case 2: Observed Action is Transmission ($a_1 \in \{2, 3\}$)

- **Detecting Channel Outcome & The Ambiguity Case:** We infer channel success if the station's new estimate equals the transmitted value. However, if the prior estimate already equaled the transmitted value ($e_1 = \text{transmitted_value}$), we cannot distinguish between a success or failure just by looking at x'_1. To prioritize correct training, we skip predictions entirely when this ambiguity occurs.

- **Predicting for $a_2 \in \{2, 3\}$ (Transmit):** If the outcome is cleanly inferred, we apply that same outcome (success or failure) to the target state. We calculate solar energy as $\delta = b'_1 - b_1 + \eta$. If successful, the target's estimate becomes the value it chose to transmit, and m'_2 resets to θ'_1. If unsuccessful, e'_2 remains e_2.

- **Predicting for $a_2 = 1$ (No Transmit):** Unlike Case 1, we **can** predict backward. Even though we transmitted, we can predict what would have happened if we didn't. The target estimate remains e_2, but the battery calculation alters to $b'_2 = \min(B, b_2 + \delta)$ because the target action does not consume the η transmission cost.

8 Analysis

8.1 Algorithm Comparison

As shown in Figure 1, Structural Knowledge Q-Learning learns significantly faster and achieves a higher average greedy return in fewer episodes compared to standard Q-Learning.

Standard Q-Learning only updates a single state-action pair for every real-world step taken in the environment. Structural Knowledge Q-Learning infers the underlying system dynamics, such as the fact that the solar energy harvested (δ_t) is completely independent of the transmission channel's success or failure. By isolating these variables, the algorithm executes pseudo-transitions, updating the Q-values of multiple related states simultaneously without having to physically visit them. This allows it to extract exponentially more information from a single time slot, resulting in drastically improved sample efficiency.

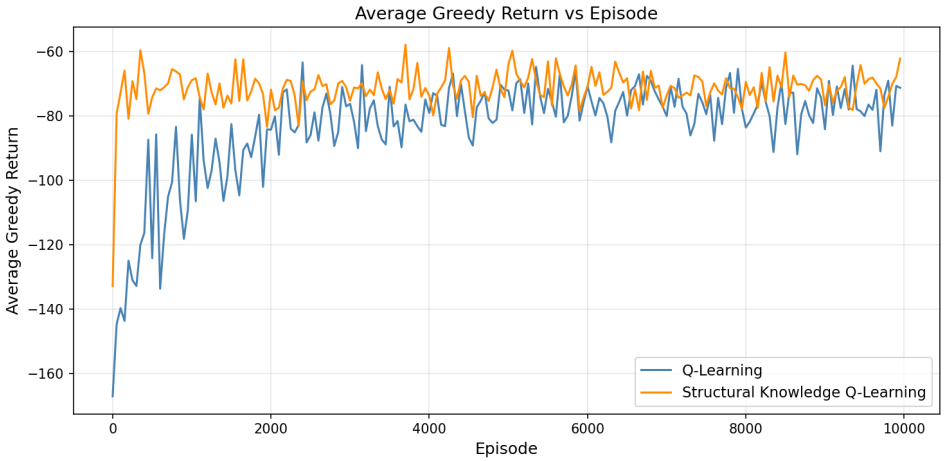

Figure 1: Average Greedy Return vs. Episode for Q-Learning and Structural Knowledge Q-Learning.

8.2 Pattern for Action 1 (Do Not Transmit)

Action 1 is the optimal action when the true air pollution (θ_t) is very close to or exactly equals the central station's current estimate (e_t).

When $\theta_t \approx e_t$, transmitting will consume $\eta = 2$ units of battery for a negligible reduction in the estimation error $|\theta_t - \hat{\theta}_t|$. Because the sensor is severely penalized for running out of battery and missing large pollution spikes later, it learns to not transmit in order to save battery for future time slots when the estimation error becomes significant.

As demonstrated in Figure 2, plotting the optimal policy for a fully charged battery ($b_t = 10$) reveals a clear diagonal band where the agent chooses Action 1. Analysis of the policy array confirms that the agent selectively chooses to conserve battery and not transmit exactly in the localized states mapping to minimal estimation error. By

remaining quiet when the station's estimate is already accurate, the agent successfully avoids redundant updates and preserves its energy limits.

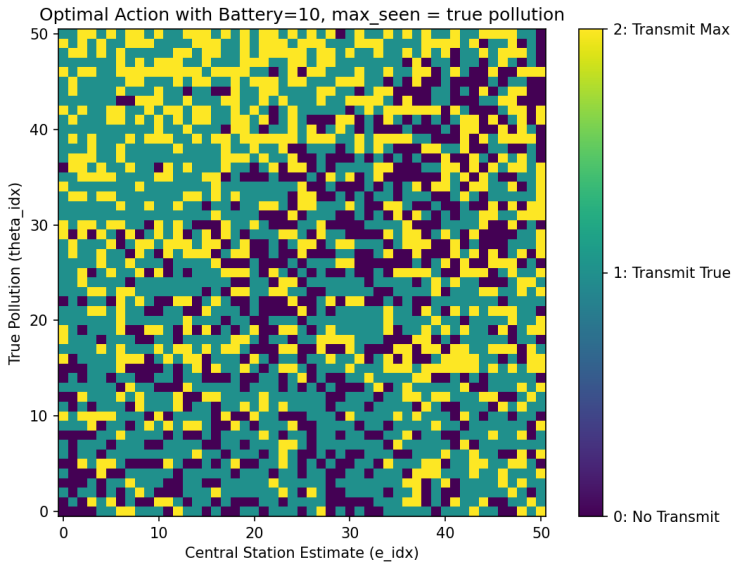

Figure 2: Optimal Action Heatmap for $b_t = 10$ and $m_t = \theta_t$. The purple diagonal band represents states where Action 1 is optimal.

8.3 Pattern for Action 3 (Transmit Max)

Action 3 is optimal when the sensor has observed a historical high spike in pollution ($m_t > \theta_t$) and the central station's current estimate (e_t) is dangerously low.

The environment's loss function penalizes the sensor 1.5 times more heavily for underestimating the pollution compared to overestimating it. Because the wireless channel is highly noisy with a low success probability ($\lambda = 0.1$), the sensor cannot guarantee that future transmissions will succeed. Therefore, when m_t is significantly larger than θ_t, it is mathematically optimal to transmit the maximum stored value rather than the true value. This acts as an insurance policy, anchoring the central station's estimate at a high value and protecting the sensor from the 1.5x underestimation penalty if subsequent transmissions fail due to channel noise or battery depletion.

Figure 3 illustrates this behavior for a scenario where the station's estimate is zero. While Action 2 (Transmit True) serves as the baseline behavior when the maximum memory closely matches the current pollution ($m_t \approx \theta_t$), the agent strategically overrides this baseline and triggers Action 3 (Transmit Max) specifically in the parameter space where a historical spike (m_t) significantly exceeds the current reality (θ_t). This localized pattern proves the agent relies on Action 3 as a dedicated, state-dependent risk-mitigation strategy.

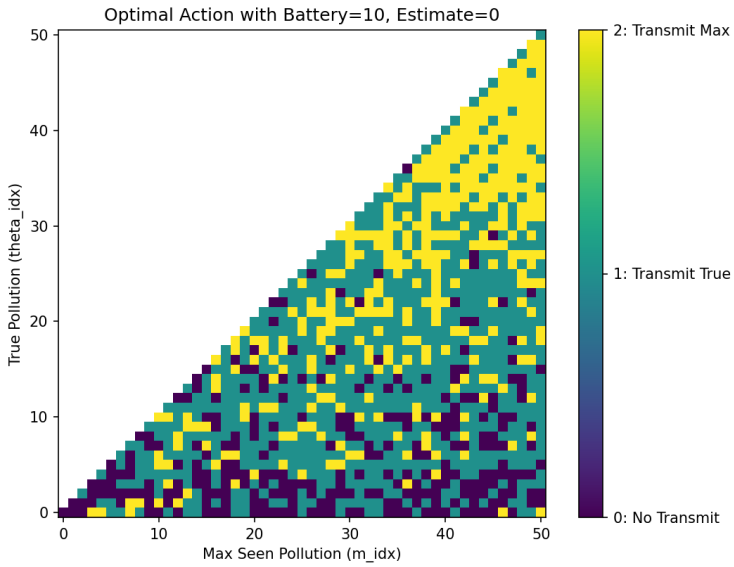

Figure 3: Optimal Action Heatmap for $b_t = 10$ and $e_t = 0$. The yellow region represents states where Action 3 is optimal, acting as insurance against underestimation penalties.